

Video Script: Orkai en 3 Niveles

VIDEO SCRIPT: Orkai en 3 Niveles — Básico, Intermedio, Avanzado

Proyecto: resume-app **Formato:** opencode terminal + slides en edición **Narración:** Español **Interfaz:** Inglés (CLI, código, prompts)

INTRO

“Si programas con inteligencia artificial y sientes que cada sesión empieza desde cero, que pierdes el control de las decisiones, o que no sabes cuánto estás gastando en tokens... este video es para ti.” “Orkai no es para vibe coders. Orkai es para ingenieros que quieren mantener el control, la calidad y la trazabilidad de lo que construyen. Corre en tu máquina. No en la nube.”

SECCIÓN 0: INSTALACIÓN Y SETUP (2-3 min)

0.1 — Descargar e instalar orkai

```
curl -fsSL https://raw.githubusercontent.com/Orkai05/installer/main/scripts/install.sh | bash
```

“Orkai se instala con un solo comando. Es un binario que baja de GitHub. Funciona en macOS, Linux y Windows. La descarga es pública, no necesitas cuenta para instalarlo.”

```
orkai version
```

“Confirmo que se instaló correctamente.”

0.2 — Obtener licencia (7 días gratis)

- Abrir getorkai.com/pricing
- Clic en **Start 7-day free trial**
- Completar checkout en Polar (\$0, sin tarjeta)

- Recibir email → clic en **Access Purchase** → copiar key (empieza con ORKAI_)

```
orkai activate ORKAI_YOUR_KEY_HERE
orkai license status
```

“7 días gratis, una máquina. Todo local en ~/.orkai/license.json. Si compras después son \$49.99 una máquina o \$99 tres dispositivos.”

0.3 — Primer arranque: configurar embeddings

```
orkai serve
```

“La primera vez entra en modo interactivo. Pregunta qué proveedor de embeddings usar. Yo elijo Voyage AI: 200 millones de tokens gratis, modelo especializado en código voyage-code-3, 32K de contexto. Si prefieres todo local, Ollama con mxbai-embed-large también funciona.”

- El wizard escribe ~/.orkai/config.yaml
- También genera credenciales en ~/.orkai/credentials

Ctrl-C para parar serve

```
orkai start
orkai status
```

“De aquí en adelante orkai start levanta el daemon en background. orkai stop lo apaga. orkai status confirma que está corriendo.”

0.4 — Conectar opencode (MCP)

```
orkai mcp-config --client opencode
```

“Orkai expone un servidor MCP local en 127.0.0.1:8787. Copio el JSON que imprime y lo pego en mi opencode.json. También funciona con Cursor, Claude Desktop, Claude Code, Codex — cualquier cliente MCP.”

0.5 — Inicializar e indexar el proyecto

```
orkai init
orkai index .
```

“Inicializo el proyecto — crea .orkai.yaml con nombre y category ID. Luego indexo todo: código Go, TypeScript, documentos markdown. Todo se convierte en entidades buscables semánticamente.”

SECCIÓN 1: INTRODUCCIÓN — ORKAI ES PARA INGENIEROS (2-3 min)

1.1 — Apertura

Abrir opencode en resume-app

Marco: Start a new session. Load project context.

AI: Llama a overview() — muestra stats: 9 estándares, 14 skills, 30 sesiones, 3 planes

“Si programas con inteligencia artificial y sientes que cada sesión empieza desde cero, que pierdes el control de las decisiones, o que no sabes cuánto estás gastando en tokens... este video es para ti.”

“Orkai no es para vibe coders. Orkai es para ingenieros que quieren mantener el control, la calidad y la trazabilidad de lo que construyen. Corre en tu máquina. No en la nube.”

“El agente llamó a overview. Una sola llamada — menos de un centavo de dólar — y ya tiene TODO el contexto del proyecto. Sesiones anteriores, estándares, skills, planes activos. No tengo que explicarle nada.”

1.2 — El balance de tokens y los 5 pasos

Slide: ANALYZE → ENCODE → PERSIST → ROUTE → ITERATE

“Esto viene de mi metodología de 5 pasos. El principio central es dividir el trabajo entre humano y LLM. Yo decido qué modelo usar para cada tarea, qué hacen los scripts determinísticos, y qué delego a la AI.”

Slide: Tabla Model Routing Strategy

Tarea	Quién	Por qué
Arquitectura, seguridad, diseño de API	Modelo fuerte	Razonamiento profundo
Buscar archivos, generar boilerplate	Modelo barato	Patrones repetitivos
Formateo, lint, tests, build	Scripts (gofmt, go vet, tsc)	Determinístico, 0 tokens

“Este balance es lo que te permite escalar. No se trata de delegar todo a la AI. Se trata de delegar lo correcto.”

SECCIÓN 2: NIVEL BÁSICO (8-10 min)

Perfil: Tú controlas todo el código. Orkai da continuidad entre sesiones y búsqueda eficiente.

2.1 — Sessions: continuidad entre días

Marco: Start a new session. I want to improve documentation on important handler functions in the backend.
First, find which handlers need better documentation – look for functions without clear comments.

AI: Usa `search_code` para encontrar handlers sin comentarios. Muestra lista con archivos y líneas.

“Fíjate. No le pedí que lea archivos uno por uno. Le pedí que ENCUENTRE funciones sin documentación. El agente usa `search_code` semánticamente. Resultado preciso en una llamada. Esto es eficiencia de tokens.”

Marco: Good. Now document the profile upload handler. Add clear Go doc comments explaining what it does, what parameters it expects, and what it returns. Follow the project's Go conventions.

AI: Lee el handler, genera los comentarios, muestra el diff.

“La AI generó los comentarios. Yo solo dirigí. ‘Documenta este handler, siguiendo las convenciones.’ Trabajo dirigido, específico.”

Marco: Now save this session. Note that I documented the profile upload handler and I still need to document the health check handler tomorrow.

AI: `session create` — guarda sesión con lo hecho y pendientes.

2.2 — Al día siguiente

Cerrar y abrir `opencode` — nueva sesión

Marco: What did I work on last time?

AI: `overview()` + `session latest` → muestra sesión anterior.

“El agente ya sabe qué hice ayer. Sabe que documenté el handler de upload y que me faltó el de health check. No empiezo desde cero.”

Marco: Continue where I left off. Document the health check handler.

AI: Encuentra el handler, agrega comentarios.

“Continuidad total. Esto es el nivel básico. No cambié mi forma de programar. Solo gané memoria entre sesiones.”

Marco: Let me see my full session history.

AI: `session list` — 30 sesiones.

“Treinta sesiones. Cada una con lo que hice, lo que quedó pendiente, y las decisiones que tomé. Esto es invaluable en proyectos de semanas o meses.”

2.3 — Búsqueda inteligente de documentos

Marco: Find the API contract standards for error response format.

AI: search_document → sección relevante del estándar, no el documento entero.

“Búsqueda semántica sobre documentos. Me devuelve solo la sección que necesito. Otra vez: eficiencia de tokens.”

2.4 — Indexar para mantener fresco

orkai index .

“Cuando hago cambios grandes, reindexo. El índice se mantiene fresco.”

Marco: Search for all places where the Success helper function is used in handlers.

“Resultados precisos del índice actualizado.”

2.5 — Cierre nivel básico

“Nivel básico resuelto. Sessions para continuidad. Búsqueda semántica para eficiencia. Indexado para mantener todo actualizado. No cambiaste tu forma de programar — solo dejaste de empezar desde cero cada día.”

SECCIÓN 3: NIVEL INTERMEDIO (9-11 min)

Perfil: Delegates features completas con estructura — Plan > Milestone > Tasks.

IMPORTANTE: En este nivel NO se mencionan workflows. Los workflows los crea el programador en el nivel avanzado.

3.1 — Planificar una feature

Marco: I want to add a new feature: PDF export for resumes. Before implementing, I need to plan it out.

Search for any existing standards and skills in orkai that would apply to PDF generation and new endpoints.

AI: Busca estándares (Backend Go+Gin, API Contract) y skills (“Add a PDF generation pipeline”, “Add a Gin endpoint”, “Add a React page”).

Marco: Based on what you found, create a plan called "Phase 3 – Resume Export". Break it into two milestones:
M1 – PDF generation pipeline in the backend. M2 – export endpoint and frontend UI.
For each task, include which standards to follow by ID and the git branch name.

AI: Crea plan → milestones → tasks con branch names y referencias a estándares.

“La AI creó el plan completo. Phase 3 con dos milestones. M1 para el pipeline de PDF, M2 para el endpoint y la UI. Cada tarea tiene su branch name y referencia los estándares por ID.”

Marco: Show me the plan structure with all tasks and their status.

AI: Árbol Plan > Milestone > Tasks, todos en pending.

3.2 — Trabajar una tarea

Marco: Save the session. Tomorrow I'll start implementing.

Día siguiente:

Marco: What's pending from yesterday's plan?

AI: overview() — muestra Phase 3 con tareas pendientes agrupadas.

“El overview me muestra las tareas pendientes. No tengo que recordar nada.”

Marco: Start working on T1 – implement the PDF generation pipeline.
Use the skill for PDF generation and follow the Backend Go+Gin standard.

AI: Crea branch feat/m1-t1-pdf-pipeline, implementa, corre gates — go build, go vet, go test.

Marco: Merge to main and mark T1 as done.

AI: Merge, task update status: "done".

“T1 completada. Si guardo la sesión y vuelvo mañana, el overview ya no muestra T1 como pendiente.”

3.3 — Tool-agnostic

“Los planes y las tareas viven en orkai, no en opencode. Si mañana abro Cursor en vez de opencode, el mismo overview me muestra las mismas tareas pendientes. La memoria es mía, no de la herramienta.”

3.4 — Cierre nivel intermedio

“Nivel intermedio. Plan > Milestone > Tasks. Estructura tu trabajo, delega features completas, mantén visibilidad. Las tareas persisten entre sesiones y entre herramientas.”

SECCIÓN 4: NIVEL AVANZADO (10-12 min)

Perfil: La AI programa dentro de tus reglas. Cada commit se audita. Los procesos repetitivos están codificados en workflows que tú diseñas.

4.1 — Standards: tus reglas de juego

Marco: List all my current standards.

AI: 9 estándares.

“Nueve estándares. Cada uno es una regla viva que evoluciona con el proyecto.”

Marco: Show me the Backend Go+Gin Conventions standard.

AI: Muestra el estándar completo — versión 10.

“Versión 10. Define cómo escribir handlers con helpers Success/InternalError, cómo manejar configuración, cómo estructurar paquetes. Cada vez que encuentro un patrón que funciona, actualizo el estándar.”

Marco: Create a new standard – all error responses must include a request_id field.

Reference the Backend Go+Gin standard for the helper functions to use.

AI: Crea el estándar.

“Nuevo estándar creado. A partir de ahora, cualquier código que genere la AI debe cumplir esta regla.”

4.2 — Review config + Git hooks

Marco: Show me the current review configuration and git hooks.

AI: Lee .orkai.yaml sección review y lefthook.yaml.

“Dos procesos: backend_review y frontend_review. Cada uno con sus estándares. Modo diff — solo revisa líneas cambiadas. Strict mode — cualquier issue rechaza el commit.”

“Pre-commit corre gofmt, go vet, oxlint y orkai review. Pre-push corre tests, build, typecheck.”

Demo del ciclo review:

Marco: Write a handler that doesn't use the Success/InternalServerError helpers, violating the Go+Gin standard. Then try to commit it.

AI: Escribe código que viola el estándar.

```
git add backend/internal/handlers/bad_handler.go
git commit -m "add handler"
# lefthook → orkai review → FAIL
```

“El commit fue rechazado. Orkai review encontró que este handler no usa los helpers que exige el estándar. El reporte está en .orkai/reports/.”

Marco: Now fix the handler to use the helpers and commit again.

```
git add backend/internal/handlers/bad_handler.go
git commit -m "add handler with standard helpers"
# lefthook → orkai review → PASS
```

“Ahora pasa. La AI no puede commitear código que viole mis reglas. Punto.”

4.3 — Workflows: procesos que tú diseñas

“Los workflows no vienen por defecto en orkai. Los creas tú basado en tu expertise. Son procesos repetitivos que codificas para que la AI los siga siempre igual.”

Marco: List my current workflows.

AI: 6 workflows — Product Owner, Architect, Feature Planner, Backend Developer, Frontend Developer, Audit 5-Step.

“Seis workflows. Cada uno lo diseñé yo para este proyecto. No vienen de fábrica.”

Marco: Show me the Backend Developer workflow.

AI: Pasos: branch → implement → go build → go vet → go test → annotate code → tag standards → merge.

“Pasos concretos. La AI no improvisa. Siempre sigue la misma secuencia.”

Marco: Show me the Architect workflow. It has two modes.

AI: Plan mode (revisa antes de construir) y Review mode (audita después de construir).

“El Architect opera en dos modos. Antes de implementar asegura cobertura de estándares. Después de implementar audita el código contra el plan, corre orkai review, arregla gaps. Esto elimina la improvisación.”

4.4 — Mi setup completo

Marco: Give me a complete overview of my project setup – config, standards, workflows, hooks.

AI: .orkai.yaml + lefthook.yaml + 9 estándares + 6 workflows + 14 skills + 30 sesiones.

“Así es como trabajo día a día. La AI programa dentro de mis reglas. Cada commit es auditado. Los procesos están codificados. Yo mantengo el control total.”

4.5 — Cierre nivel avanzado

“Nivel avanzado. No es magia — es ingeniería. Tú defines las reglas como estándares. Conectas la auditoría a git hooks. Diseñas workflows para tus procesos repetitivos. La AI se convierte en un miembro de tu equipo que sigue tus reglas.”

SECCIÓN 5: CIERRE GENERAL (1-2 min)

5.1 — Resumen de los 3 niveles

“Tres niveles, una herramienta:

Básico — sessions y búsqueda semántica. Ganas continuidad y eficiencia sin cambiar tu flujo.

Intermedio — plan > milestone > tasks. Estructura, delegación, trazabilidad.

Avanzado — standards + workflows + git hooks. La AI programa dentro de tus reglas.

Todo corre en tu máquina. Tool-agnostic. Open source.”

5.2 — Call to action

“Este proyecto, resume-app, es open source. Clónalo, instala orkai, y empieza por el nivel que te acomode.

Si te gustó, suscríbete. El workshop completo del método de los 5 pasos está en la descripción.”

APÉNDICE: PROMPTS POR NIVEL

Nivel Básico

Acción	Prompt
Iniciar	Start a new session. Load project context.
Buscar	Find functions in the backend that need better documentation.
Editar	Document the profile upload handler with Go doc comments. Follow the project conventions.
Guardar	Save this session. I documented X, pending Y.
Continuar	What did I work on last time?
Reanudar	Continue where I left off. Document the health check handler.
Buscar docs	Find the API contract standard for error response format.
Indexar	Re-index the project to refresh everything.

Nivel Intermedio

Acción	Prompt
Planear	I want to add PDF export. Find existing standards and skills that apply.
Crear plan	Create a plan Phase 3 with two milestones. Include branch names and standards per task.
Ver plan	Show me the plan structure with all tasks and status.
Continuar	What's pending from yesterday's plan?
Ejecutar	Start T1 – implement PDF pipeline using the PDF skill and Go+Gin standard.
Completar	Merge to main and mark T1 as done.

Nivel Avanzado

Acción	Prompt
Listar std	List all my current standards.
Ver std	Show me the Backend Go+Gin Conventions standard.
Crear std	Create a new standard: error responses must include request_id.
Ver review	Show me the current review configuration.
Ver hooks	Show me the git hooks configuration.
Romper std	Write a handler that violates the Go+Gin standard so I can demo the review gate.
Corregir	Fix the handler to use the helpers and commit again.
Listar wf	List my current workflows.
Ver wf	Show me the Backend Developer workflow.
Setup todo	Give me a complete overview of my project setup – config, standards, workflows, hooks.

NOTAS DE PRODUCCIÓN

Slides a insertar en edición: 1. Los 5 pasos — ANALYZE → ENCODE → PERSIST → ROUTE → ITERATE 2. Tabla Model Routing Strategy (fuerte vs barato vs scripts) 3. Tabla comparativa de los 3 niveles

Escenarios a preparar antes de grabar: 1. Tener una sesión guardada de antemano para mostrar What did I work on last time? 2. Tener un plan con tareas pre-creado para el nivel intermedio 3. Tener un handler “malo” listo para la demo de review 4. Tener .orkai/reports/ con al menos un reporte previo

Comandos a tener en el historial: - orkai version, orkai start, orkai status, orkai index . - orkai mcp-config --client opencode - git commit que falle por review